Language Modeling

Comprehensive Cheatsheet

Machine Intelligence - Andrés Masegosa - Fall 2025

Contents

1 What is Language Modeling? 3

1.1 The Big Picture . . . 3

2 Probability Foundations 4

2.1 Chain Rule of Probability . . . 4

3 The Markov Assumption 5

4 N-gram Models 6

4.1 Overview . . . 6

4.2 Unigram Model . . . 6

4.3 Bigram Model . . . 7

4.4 Trigram Model . . . 7

4.5 General N-gram . . . 7

5 Maximum Likelihood Estimation (MLE) 8

5.1 The Core Idea . . . 8

5.2 Detailed Example: The "Sam" Corpus . . . 10

5.3 Handling Unknown Words . . . 13

5.4 The Zero Probability Problem . . . 13

6 Smoothing 14

6.1 The Intuition . . . 14

6.2 Add-1 Smoothing (Laplace Smoothing) . . . 14

7 Evaluation: Perplexity 16

7.1 Why Evaluate? . . . 16

7.2 The Shannon Game . . . 17

7.3 Perplexity Definition . . . 17

7.4 Using Log Probabilities . . . 20

8 Classification Evaluation Metrics **20**
8.1 Confusion Matrix 20
8.2 Accuracy 21
8.3 Precision and Recall 22
8.4 F1-Score 23

9 Step-by-Step Guide to Solving Exercises **24**
9.1 Exercise Type 1: Computing Unigram Counts 24
9.2 Exercise Type 2: Computing MLE for Bigrams 24
9.3 Exercise Type 3: Computing MLE for Trigrams 26
9.4 Exercise Type 4: Computing Sentence Probability 27

10 Word Meaning and Semantics **28**
10.1 The Problem: N-grams Don't Understand Meaning 28
10.2 Understanding Word Components 29
10.3 Synonyms: Words with Similar Meanings 30
10.4 Word Relatedness vs. Similarity 32
10.5 Vector Semantics: The Solution! 34

11 Quick Reference: Key Formulas **37**

12 Tips for Exam Success **39**

1 What is Language Modeling?

1.1 The Big Picture

What is a Language Model?

Definition: A language model is a probabilistic model that assigns probabilities to sequences of words.
Two Main Tasks:

1. **Sentence Probability**: Compute $P(W) = P(w_1, w_2, w_3, \ldots, w_n)$
   - "What is the probability of this entire sentence?"
   - Example: $P(\text{“I love programming”})$
2. **Next Word Prediction**: Compute $P(w_n | w_1, w_2, \ldots, w_{n-1})$
   - "Given these words, what is the probability of the next word?"
   - Example: $P(\text{“programming”} | \text{“I love”})$

Why Do We Need This?

- **Machine Translation**: "high winds tonight" is more likely than "large winds tonight"
- **Spell Correction**: "fifteen minutes from" is more likely than "fifteen minuets from"
- **Speech Recognition**: "I saw a van" is more likely than "eyes awe of an"
- **Text Generation**: Predict next words to generate text
- **Question Answering, Summarization**, and many more!

2 Probability Foundations

2.1 Chain Rule of Probability

Chain Rule - The Foundation

Basic Conditional Probability:

$$P(B|A) = \frac{P(A \cap B)}{P(A)} = \frac{P(A, B)}{P(A)}$$

Plain English: "The probability of B given A equals the probability of both A and B happening, divided by the probability of A happening."
Rearranging:

$$P(A, B) = P(A) \times P(B|A)$$

Plain English: "The probability of both A and B happening equals the probability of A happening times the probability of B happening given that A happened."
For More Variables:

$$P(A, B, C, D) = P(A) \times P(B|A) \times P(C|A, B) \times P(D|A, B, C)$$

General Chain Rule:

$$P(x_1, x_2, x_3, \ldots, x_n) = P(x_1) \times P(x_2|x_1) \times P(x_3|x_1, x_2) \times \cdots \times P(x_n|x_1, \ldots, x_{n-1})$$

Plain English: "Break down a joint probability into a product of conditional probabilities, where each word depends on all previous words."

Chain Rule Example

Sentence: "its water is so transparent"
Using Chain Rule:

$$\begin{aligned} P(\text{its, water, is, so, transparent}) = \\ P(\text{its})\times \\ P(\text{water}|\text{its})\times \\ P(\text{is}|\text{its water})\times \\ P(\text{so}|\text{its water is})\times \\ P(\text{transparent}|\text{its water is so}) \end{aligned}$$

Step-by-Step in Plain English:

1. Probability of starting with "its"
2. THEN, probability of "water" coming after "its"
3. THEN, probability of "is" coming after "its water"
4. THEN, probability of "so" coming after "its water is"
5. THEN, probability of "transparent" coming after "its water is so"

The Problem with Chain Rule

How Do We Estimate These Probabilities?
The naive approach would be:

$$P(\text{the}|\text{its water is so transparent that}) = \frac{\text{Count(its water is so transparent that the)}}{\text{Count(its water is so transparent that)}}$$

Why This Fails:

- Too many possible sentences! We'd never see most sequences in our training data
- The longer the sequence, the less likely we've seen it before
- We need a clever approximation...

3 The Markov Assumption

Making Language Modeling Practical

The Key Insight: Maybe we don't need ALL previous words to predict the next word. Maybe just the last few words are enough!
Markov Assumption: Approximate the probability of a word based only on the previous k words:

$$P(w_i|w_1, w_2, \ldots, w_{i-1}) \approx P(w_i|w_{i-k}, \ldots, w_{i-1})$$

Plain English: "To predict the next word, we only look at the last k words, not the entire history."
Examples:

- $P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$ (look at last 1 word)
- $P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{transparent that})$ (look at last 2 words)

Full Sentence Probability:

$$P(w_1, w_2, \ldots, w_n) \approx \prod_{i=1}^{n} P(w_i|w_{i-k}, \ldots, w_{i-1})$$

4 N-gram Models

4.1 Overview

What are N-grams?

N-gram: A sequence of N consecutive words.
Types:

- **Unigram (1-gram)**: Single word - "I", "love", "programming"
- **Bigram (2-gram)**: Two consecutive words - "I love", "love programming"
- **Trigram (3-gram)**: Three consecutive words - "I love programming"
- **N-gram**: N consecutive words

N-gram Language Model: Uses the Markov assumption to simplify probability calculation by looking at only the previous $N-1$ words.

4.2 Unigram Model

Unigram Model - The Simplest Model

Assumption: Each word is independent of all other words. No context!
Formula:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i)$$

Plain English: "The probability of a sentence is just the product of the individual word probabilities."
Example: $P(\text{“I love programming”})$

$$P(\text{“I love programming”}) = P(\text{“I”}) \times P(\text{“love”}) \times P(\text{“programming”})$$

Limitations:

- Ignores all context
- Generates nonsensical sentences
- Example generated text: "fifth, an, of, futures, the, an, incorporated, a..."

When to Use: Rarely used alone, but useful as a baseline or for simple tasks.

4.3 Bigram Model

Bigram Model - Looking at One Previous Word

Assumption: Each word depends only on the immediately preceding word.
Formula:

$$P(w_i|w_1, w_2, \ldots, w_{i-1}) \approx P(w_i|w_{i-1})$$

Full Sentence:

$$P(w_1, w_2, \ldots, w_n) \approx \prod_{i=1}^{n} P(w_i|w_{i-1})$$

Plain English: "To predict the next word, look at only the previous word."
Example: $P(\text{“I love programming”})$

$$\begin{aligned} P(\text{“I love programming”}) = & P(\text{“I”}) \times \\ & P(\text{“love”}|\text{“I”}) \times \\ & P(\text{“programming”}|\text{“love”}) \end{aligned}$$

Better than Unigram: Captures local context (one word back).

4.4 Trigram Model

Trigram Model - Looking at Two Previous Words

Assumption: Each word depends on the two immediately preceding words.
Formula:

$$P(w_i|w_1, w_2, \ldots, w_{i-1}) \approx P(w_i|w_{i-2}, w_{i-1})$$

Full Sentence:

$$P(S) = \prod_{i=1}^{n} P(w_i|w_{i-2}, w_{i-1})$$

Plain English: "To predict the next word, look at the previous TWO words."
Better than Bigram: Captures more context, but requires more training data.

4.5 General N-gram

General N-gram Model

For N-grams:

$$P(S) = \prod_{i=1}^{n} P(w_i|w_{i-N+1}, \ldots, w_{i-1})$$

Practical Considerations:

- **Space Complexity**: $O(|V|^n)$ where V is vocabulary size
- **Typical N values**: 2 (bigram), 3 (trigram), 4 (4-gram)
- **Trade-off**:
  - Larger N = More context = Better predictions
  - Larger N = More parameters = More data needed = Higher complexity

5 Maximum Likelihood Estimation (MLE)

5.1 The Core Idea

How Do We Estimate Probabilities?

Maximum Likelihood Estimation (MLE): Estimate probabilities by counting occurrences in training data.
Core Principle: "The probability of something is simply how often it appears in our data."
For Bigrams:

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Plain English:

- **Numerator**: Count how many times the word pair (w_{i-1}, w_i) appears
- **Denominator**: Count how many times the first word w_{i-1} appears
- **Result**: Fraction of times w_i follows w_{i-1}

Shorthand: $C(x)$ means "Count of x"

5.2 Detailed Example: The "Sam" Corpus

Complete MLE Bigram Example

Training Corpus:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

Vocabulary: {I, am, Sam, do, not, like, green, eggs, and, ham}
Special Tokens:

- `<s>`: Start of sentence
- `</s>`: End of sentence

Step 1: Count All Bigrams
Create a count matrix (rows = first word, columns = second word):

	I	am	Sam	do	not	like	green	eggs	and	ham	<s>	</s>
I	0	2	0	1	0	0	0	0	0	0	0	0
am	0	0	1	0	0	0	0	0	0	0	0	1
Sam	1	0	0	0	0	0	0	0	0	0	0	1
do	0	0	0	0	1	0	0	0	0	0	0	0
not	0	0	0	0	0	1	0	0	0	0	0	0
like	0	0	0	0	0	0	1	0	0	0	0	0
green	0	0	0	0	0	0	0	1	0	0	0	0
eggs	0	0	0	0	0	0	0	0	1	0	0	0
and	0	0	0	0	0	0	0	0	0	1	0	0
ham	0	0	0	0	0	0	0	0	0	0	0	1
<s>	2	0	1	0	0	0	0	0	0	0	0	0

Step 2: Calculate Probabilities
For each bigram, divide by the row sum (count of first word):
Examples:

$$P(\mathrm{I}|\texttt{<s>}) = \frac{C(\texttt{<s>}, \mathrm{I})}{C(\texttt{<s>})} = \frac{2}{3} = 0.67$$
$$P(\mathrm{Sam}|\texttt{<s>}) = \frac{C(\texttt{<s>}, \mathrm{Sam})}{C(\texttt{<s>})} = \frac{1}{3} = 0.33$$
$$P(\mathrm{am}|\mathrm{I}) = \frac{C(\mathrm{I}, \mathrm{am})}{C(\mathrm{I})} = \frac{2}{3} = 0.67$$
$$P(\mathrm{do}|\mathrm{I}) = \frac{C(\mathrm{I}, \mathrm{do})}{C(\mathrm{I})} = \frac{1}{3} = 0.33$$
$$P(\mathrm{Sam}|\mathrm{am}) = \frac{C(\mathrm{am}, \mathrm{Sam})}{C(\mathrm{am})} = \frac{1}{2} = 0.50$$
$$P(\texttt{</s>}|\mathrm{Sam}) = \frac{C(\mathrm{Sam}, \texttt{</s>})}{C(\mathrm{Sam})} = \frac{1}{2} = 0.50$$

Step 3: Calculate Sentence Probability
For sentence: `<s> I am Sam </s>`

$$\begin{aligned} P(S) &= P(\mathrm{I}|\texttt{<s>}) \times P(\mathrm{am}|\mathrm{I}) \times P(\mathrm{Sam}|\mathrm{am}) \times P(\texttt{</s>}|\mathrm{Sam}) \\ &= 0.67 \times 0.67 \times 0.50 \times 0.50 \\ &= 0.112 \end{aligned}$$

Complete MLE Trigram Example

Same Training Corpus:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

Trigram MLE Formula:

$$P(w_3|w_1, w_2) = \frac{C(w_1, w_2, w_3)}{C(w_1, w_2)}$$

Step 1: Count All Trigrams

List all trigrams (3-word sequences) from the corpus:

How to Read This Table:

- **Column 1 - Trigram**: The 3-word sequence we found in the corpus
- **Column 2 - Count**: How many times this exact 3-word sequence appears in all sentences
- **Column 3 - Bigram Context**: The first 2 words of the trigram (this is the "given" part in $P(w_3|w_1, w_2)$)
- **Column 4 - Bigram Count**: How many times the first 2 words appear together (this is the denominator)

Example Row: (<s>, I, am) — 1 — (<s>, I) — 2

"The sequence '<s> I am' appears 1 time. To calculate its probability, we divide by how often '<s> I' appears, which is 2 times. So $P(am|$<s>$, I) = \frac{1}{2}$."

Trigram	Count	Bigram Context	Bigram Count
(<s>, I, am)	1	(<s>, I)	2
(<s>, Sam, I)	1	(<s>, Sam)	1
(<s>, I, do)	1	(<s>, I)	2
(I, am, Sam)	1	(I, am)	2
(I, am, </s>)	1	(I, am)	2
(am, Sam, </s>)	1	(am, Sam)	1
(Sam, I, am)	1	(Sam, I)	1
(I, do, not)	1	(I, do)	1
(do, not, like)	1	(do, not)	1
(not, like, green)	1	(not, like)	1
(like, green, eggs)	1	(like, green)	1
(green, eggs, and)	1	(green, eggs)	1
(eggs, and, ham)	1	(eggs, and)	1
(and, ham, </s>)	1	(and, ham)	1

Important: (<s>, I, am) has count **1**, not 2! Why?

- Sentence 1: `<s> I am...` → (<s>, I, am) ✓
- Sentence 2: `<s> Sam I...` → (<s>, Sam, I) — different trigram!
- Sentence 3: `<s> I do...` → (<s>, I, do) — different trigram!

Step 2: Calculate Probabilities

For each trigram, divide the trigram count by the bigram context count:

Examples:

$$P(\text{am}|\texttt{<s>}, \text{I}) = \frac{C(\texttt{<s>}, \text{I}, \text{am})}{C(\texttt{<s>}, \text{I})} = \frac{1}{2} = 0.50$$

$$P(\text{do}|\texttt{<s>}, \text{I}) = \frac{C(\texttt{<s>}, \text{I}, \text{do})}{C(\texttt{<s>}, \text{I})} = \frac{1}{2} = 0.50$$

$$P(\text{I}|\texttt{<s>}, \text{Sam}) = \frac{C(\texttt{<s>}, \text{Sam}, \text{I})}{C(\texttt{<s>}, \text{Sam})} = \frac{1}{1} = 1.00$$

$$P(\text{Sam}|\text{I}, \text{am}) = \frac{C(\text{I}, \text{am}, \text{Sam})}{C(\text{I}, \text{am})} = \frac{1}{2} = 0.50$$

$$P(\texttt{</s>}|\text{I}, \text{am}) = \frac{C(\text{I}, \text{am}, \texttt{</s>})}{C(\text{I}, \text{am})} = \frac{1}{2} = 0.50$$

$$P(\text{am}|\text{Sam}, \text{I}) = \frac{C(\text{Sam}, \text{I}, \text{am})}{C(\text{Sam}, \text{I})} = \frac{1}{1} = 1.00$$

$$P(\texttt{</s>}|\text{am}, \text{Sam}) = \frac{C(\text{am}, \text{Sam}, \texttt{</s>})}{C(\text{am}, \text{Sam})} = \frac{1}{1} = 1.00$$

Step 3: Calculate Sentence Probability

For sentence: `<s> I am Sam </s>`

How to Read This: We multiply the probability of each word, given the previous words.

Word	Question We Ask	Formula	Value	Type
I	"After <s>, what's the chance of I?"	$P(\text{I}\|\texttt{<s>})$	$\frac{2}{3} = 0.67$	Bigram
am	"After <s> I, what's the chance of am?"	$P(\text{am}\|\texttt{<s>}, \text{I})$	$\frac{1}{2} = 0.50$	Trigram
Sam	"After I am, what's the chance of Sam?"	$P(\text{Sam}\|\text{I}, \text{am})$	$\frac{1}{2} = 0.50$	Trigram
</s>	"After am Sam, what's the chance of ending?"	$P(\texttt{</s>}\|\text{am}, \text{Sam})$	$\frac{1}{1} = 1.00$	Trigram

Why does "I" use a Bigram, not a Trigram?

Trigrams need **2 previous words** to make a prediction. But for the first word "I":

- We only have <s> before it (that's just 1 word)
- We can't use $P(\text{I}|?, \texttt{<s>})$ because there's nothing before <s|
- So we "back off" to a bigram: $P(\text{I}|\texttt{<s>})$

Alternative: Some textbooks add TWO start tokens: `<s> <s> I am Sam </s>`. Then you can use $P(\text{I}|\texttt{<s>}, \texttt{<s>})$ as a trigram.

Final Calculation: Multiply all probabilities together:

$$P(S) = 0.67 \times 0.50 \times 0.50 \times 1.00 = 0.167$$

Plain English: "There's a 16.7% chance of generating this exact sentence."

Comparison with Bigram:

- **Bigram probability**: $P(S) = 0.112$
- **Trigram probability**: $P(S) = 0.167$
- Trigram gives *higher* probability because context (am, Sam) uniquely determines </s>

5.3 Handling Unknown Words

The Unknown Word Problem

Problem: What if we encounter a word not in our vocabulary during testing?
Example: Our vocabulary is {I, am, Sam, do, not, like, green, eggs, and, ham}
Test sentence: "I am John"
Problem: $P(\text{John}|\text{am}) = \frac{0}{2} = 0 \rightarrow$ entire sentence probability $= 0$!
Solutions:

1. **Closed Vocabulary Assumption**:
   - Assume test set only contains words from vocabulary
   - Not realistic for real applications
2. **Open Vocabulary Assumption** (Better):
   - Add a special token `<UNK>` to vocabulary
   - In training: Replace rare/unseen words with `<UNK>`
   - In testing: Replace unseen words with `<UNK>`
   - Estimate probabilities for `<UNK>` like any other word

5.4 The Zero Probability Problem

Major Problem with MLE

Issue: If a bigram never appears in training, its probability is 0.
Example:

- We never saw "Sam likes" in training
- $P(\text{likes}|\text{Sam}) = \frac{0}{\text{something}} = 0$

Why This is Bad:

- One zero probability makes entire sentence probability $= 0$
- Many reasonable sentences will have probability 0
- Model too rigid - can't handle unseen but valid sequences

Real-World Example (Shakespeare corpus):

- Vocabulary: 29,066 word types
- Only 0.04% of possible bigrams occurred
- 99.96% of bigrams have probability 0!

Solution: Smoothing (next section)

6 Smoothing

6.1 The Intuition

Why Smoothing?

Problem: MLE assigns probability 0 to unseen events.
Solution: "Steal" some probability mass from seen events and give it to unseen events.
Analogy: Like a progressive tax system

- Take a little from the rich (frequent events)
- Give to the poor (rare/unseen events)
- Everyone gets something!

Visual Example:
Before Smoothing:

```
allegations: *******  (3 occurrences)
reports:     ****     (2 occurrences)
claims:      **       (1 occurrence)
request:     **       (1 occurrence)
attack:               (0 occurrences) <- ZERO!
outcome:              (0 occurrences) <- ZERO!
```

After Smoothing:

```
allegations: ******   (2.5 redistributed)
reports:     ***      (1.5 redistributed)
claims:      *        (0.5 redistributed)
request:     *        (0.5 redistributed)
attack:      *        (0.5 added!)
outcome:     *        (0.5 added!)
```

6.2 Add-1 Smoothing (Laplace Smoothing)

Add-1 (Laplace) Smoothing

Idea: Pretend we saw each word one more time than we actually did.
For Bigrams:
MLE (Original):

$$P_{\text{MLE}}(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Add-1 Smoothed:

$$P_{\text{Add-1}}(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

where V = vocabulary size (number of unique words)
Plain English:

- Add 1 to numerator: "Pretend we saw this bigram one more time"
- Add V to denominator: "We added 1 for each possible word in vocabulary"

Key Point: Now NO probability is zero! Every word gets at least $\frac{1}{C(w_{i-1})+V}$ probability.

Add-1 Smoothing Example

Setup: Vocabulary size $V = 8$ (I, am, Sam, do, not, like, green, eggs, and, ham, excluding special tokens)
Let's smooth some probabilities:
Before Add-1 (MLE):

$$P_{\text{MLE}}(\text{am}|\text{I}) = \frac{2}{3} = 0.67$$
$$P_{\text{MLE}}(\text{Sam}|\text{I}) = \frac{0}{3} = 0 \quad \leftarrow \text{ZERO!}$$

After Add-1:

$$P_{\text{Add-1}}(\text{am}|\text{I}) = \frac{2+1}{3+8} = \frac{3}{11} = 0.27$$
$$P_{\text{Add-1}}(\text{Sam}|\text{I}) = \frac{0+1}{3+8} = \frac{1}{11} = 0.09 \quad \leftarrow \text{No longer zero!}$$

Observations:

- Frequent events: Probability decreased ($0.67 \rightarrow 0.27$)
- Unseen events: Probability increased ($0 \rightarrow 0.09$)
- All probabilities still sum to 1

Complete Add-1 Smoothing: Berkeley Restaurant Corpus

Vocabulary: 8 words: i, want, to, eat, chinese, food, lunch, spend
Original Counts (from corpus):

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211

After Add-1 Smoothing (add 1 to each count):

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212

Probabilities (divide by row sum + V):
For word "i" (appears 843 times total, after smoothing: 843 + 8 = 851):

$$P(\text{want}|\text{i}) = \frac{828}{851} = 0.97$$
$$P(\text{to}|\text{i}) = \frac{1}{851} = 0.0012 \quad \leftarrow \text{Was 0, now non-zero!}$$

Limitations of Add-1 Smoothing

Problem: Add-1 often too aggressive for N-grams!
Why:

- Gives too much probability to unseen events
- Takes too much from seen events
- Works better for other tasks (e.g., text classification)

Better Alternatives (not covered in detail here):

- Add-k smoothing (add a fraction instead of 1)
- Good-Turing smoothing
- Kneser-Ney smoothing (state-of-the-art for N-grams)
- Interpolation methods

7 Evaluation: Perplexity

7.1 Why Evaluate?

Two Types of Evaluation

1. **Extrinsic Evaluation (In-Vivo)**:
   - Test the language model in a real application
   - Example: Use LM in machine translation, measure translation quality
   - **Pros**: Directly measures usefulness
   - **Cons**: Expensive, time-consuming, task-specific
2. **Intrinsic Evaluation (In-Vitro)** - Perplexity:
   - Test the language model on its own terms
   - Measure: How well does it predict a test set?
   - **Pros**: Fast, cheap, general-purpose metric
   - **Cons**: Doesn't always correlate with task performance

Focus: We'll use Perplexity as our main intrinsic metric.

7.2 The Shannon Game

Intuition: Predicting the Next Word

The Shannon Game: How well can you predict the next word?
Examples:

- "Once upon a ____" → Most likely: "time" (high probability)
- "That is a picture of a ____" → Could be many things

Key Insight: A good language model assigns high probability to the word that actually comes next.
Formalization:

- Bad LM: Assigns low probability to actual test sentences
- Good LM: Assigns high probability to actual test sentences

7.3 Perplexity Definition

Perplexity - The Inverse Probability

Definition: Perplexity is the inverse probability of the test set, normalized by the number of words.
Formula:

$$\mathrm{PP}(W) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}}$$

Alternative Form:

$$\mathrm{PP}(W) = \sqrt[N]{\frac{1}{P(w_1, w_2, \ldots, w_N)}}$$

Plain English:

- Take the probability of the sentence
- Take its inverse (flip it: high prob → low value, low prob → high value)
- Take the N-th root (normalize by length)

Interpretation:

- **Lower perplexity** = Better model (higher probability assigned)
- **Higher perplexity** = Worse model (lower probability assigned)
- Range: $[1, \infty)$
- Minimum value 1 (perfect prediction)

Perplexity for Different N-gram Models

General Chain Rule Form:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_1, \ldots, w_{i-1})}}$$

For Bigram Model:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}}$$

Plain English: "Perplexity is the geometric mean of the inverse probabilities of each word given its context."
Why Geometric Mean? Because we're multiplying probabilities, geometric mean makes sense for averaging.

Perplexity Example 1: Simple Language

Deterministic Language: $L = \{\text{red}, \text{blue}, \text{green}\}$
Branching Factor: 3 (any word can follow any word)
Model A: Uniform distribution - each word equally likely ($\frac{1}{3}$)
Test Set: "red red red red blue" (5 words)
Calculate Perplexity:

$$\begin{aligned} P(\text{test set}) &= P(\text{red}) \times P(\text{red}) \times P(\text{red}) \times P(\text{red}) \times P(\text{blue}) \\ &= \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \\ &= \left(\frac{1}{3}\right)^5 \end{aligned}$$

Perplexity:

$$\begin{aligned} \mathrm{PP} &= P(\text{test set})^{-\frac{1}{5}} \\ &= \left[\left(\frac{1}{3}\right)^5\right]^{-\frac{1}{5}} \\ &= \left(\frac{1}{3}\right)^{-1} \\ &= 3 \end{aligned}$$

Interpretation: On average, the model is "3-way confused" about the next word.

Perplexity Example 2: Better Model

Model B: Non-uniform distribution based on actual frequencies

- $P(\text{red}) = 0.8$
- $P(\text{green}) = 0.1$
- $P(\text{blue}) = 0.1$

Same Test Set: "red red red red blue"
Calculate Probability:

$$\begin{aligned} P(\text{test set}) &= 0.8 \times 0.8 \times 0.8 \times 0.8 \times 0.1 \\ &= 0.04096 \end{aligned}$$

Perplexity:

$$\begin{aligned} \text{PP} &= (0.04096)^{-\frac{1}{5}} \\ &= (0.04096)^{-0.2} \\ &= 1.89 \end{aligned}$$

Comparison:

- Model A (uniform): PP = 3.0
- Model B (trained): PP = 1.89
- **Model B is better!** (lower perplexity)

Real-World Perplexity: Wall Street Journal

Dataset:

- Vocabulary: 19,979 words
- Training set: 38 million words
- Test set: 1.5 million words

Results:

Model	Perplexity
Unigram	962
Bigram	170
Trigram	109

Observations:

- More context (higher N) = Lower perplexity = Better model
- Unigram terrible (essentially random guessing from 962 options)
- Trigram best (on average confused among 109 options)

7.4 Using Log Probabilities

Practical Implementation: Log Space

Problem: Multiplying many small probabilities causes numerical underflow.
Example: $(0.001)^{1000} \approx 10^{-3000} \rightarrow$ Too small for computers!
Solution: Work in log space
Original Perplexity:

$$\text{PP}(S) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}}$$

Log-Space Perplexity:

$$\text{PP}(S) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(w_i|w_1, \ldots, w_{i-1})\right)$$

Benefits:

- Multiplication becomes addition: $\log(a \times b) = \log(a) + \log(b)$
- No underflow issues
- Faster computation

8 Classification Evaluation Metrics

8.1 Confusion Matrix

Confusion Matrix for Binary Classification

Setup: Binary classification (Positive vs Negative)
Four Outcomes:

		Predicted	
		Positive	Negative
Actual	Positive	**TP** (True Positive)	**FN** (False Negative)
	Negative	**FP** (False Positive)	**TN** (True Negative)

Definitions:

- **True Positive (TP)**: Correctly predicted positive
- **True Negative (TN)**: Correctly predicted negative
- **False Positive (FP)**: Incorrectly predicted positive (Type I Error)
- **False Negative (FN)**: Incorrectly predicted negative (Type II Error)

Example: Spam Detection

Task: Classify emails as Spam (Positive) or Not Spam (Negative)
Test Set: 100 emails
Results:

- 40 spam emails correctly classified as spam (TP)
- 45 not-spam emails correctly classified as not-spam (TN)
- 5 not-spam emails incorrectly classified as spam (FP)
- 10 spam emails incorrectly classified as not-spam (FN)

Confusion Matrix:

		Predicted	
		Spam	Not Spam
Actual	Spam	40 (TP)	10 (FN)
	Not Spam	5 (FP)	45 (TN)

8.2 Accuracy

Accuracy

Definition: Proportion of correctly classified instances out of all instances.
Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Plain English: "What fraction of predictions were correct?"
Example (from spam detection):

$$\text{Accuracy} = \frac{40 + 45}{40 + 45 + 5 + 10} = \frac{85}{100} = 0.85 = 85\%$$

When Accuracy is Misleading:

- Imbalanced datasets!
- Example: 99% emails are not spam, 1% are spam
- Classifier that always predicts "not spam": 99% accuracy!
- But catches 0% of spam → useless!

Solution: Use precision and recall for imbalanced data.

8.3 Precision and Recall

Precision

Definition: Of all positive predictions, how many were correct?
Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Plain English: "When the model says positive, how often is it right?"
Example:

$$\text{Precision} = \frac{40}{40 + 5} = \frac{40}{45} = 0.89 = 89\%$$

Interpretation: 89% of emails classified as spam were actually spam.
When to Care:

- When false positives are costly
- Example: Don't want important emails marked as spam

Recall (Sensitivity)

Definition: Of all actual positives, how many did we catch?
Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Plain English: "Of all real positive cases, how many did we find?"
Example:

$$\text{Recall} = \frac{40}{40 + 10} = \frac{40}{50} = 0.80 = 80\%$$

Interpretation: We caught 80% of all spam emails.
When to Care:

- When false negatives are costly
- Example: Cancer detection - can't miss any cases!

8.4 F1-Score

F1-Score - Harmonic Mean

Definition: Harmonic mean of precision and recall. Balances both metrics.
Formula:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Alternative Form:

$$F_1 = \frac{2 \times TP}{2 \times TP + FP + FN}$$

Plain English: "Single score that balances precision and recall."
Example:

$$F_1 = 2 \times \frac{0.89 \times 0.80}{0.89 + 0.80} = 2 \times \frac{0.712}{1.69} = 0.84 = 84\%$$

Why Harmonic Mean?

- Punishes extreme values more than arithmetic mean
- If either precision OR recall is low, F1 will be low
- Forces model to balance both metrics

Complete Metrics Example

Confusion Matrix:

		Predicted	
		Positive	Negative
Actual	Positive	30 (TP)	20 (FN)
	Negative	10 (FP)	40 (TN)

Calculate All Metrics:
Accuracy:

$$\text{Accuracy} = \frac{30 + 40}{30 + 40 + 10 + 20} = \frac{70}{100} = 0.70 = 70\%$$

Precision:

$$\text{Precision} = \frac{30}{30 + 10} = \frac{30}{40} = 0.75 = 75\%$$

Recall:

$$\text{Recall} = \frac{30}{30 + 20} = \frac{30}{50} = 0.60 = 60\%$$

F1-Score:

$$F_1 = 2 \times \frac{0.75 \times 0.60}{0.75 + 0.60} = 2 \times \frac{0.45}{1.35} = 0.67 = 67\%$$

Interpretation:

- 70% overall accuracy
- When model says positive, it's right 75% of the time
- Model catches 60% of actual positives
- Balanced F1 score of 67%

9 Step-by-Step Guide to Solving Exercises

9.1 Exercise Type 1: Computing Unigram Counts

Task: Given a corpus, count each word.

Corpus Example:

```
1. "I love machine learning"
2. "I love deep learning"
3. "Deep learning is great"
4. "Machine learning is fun"
```

Step-by-Step:

Step 1: List all unique words (vocabulary)

- Vocabulary: {I, love, machine, learning, deep, is, great, fun}

Step 2: Count each word across all sentences

- I: 2
- love: 2
- machine: 2
- learning: 4
- deep: 2
- is: 2
- great: 1
- fun: 1

Step 3: Calculate unigram probabilities

- Total words: 16
- $P(\text{I}) = \frac{2}{16} = 0.125$
- $P(\text{learning}) = \frac{4}{16} = 0.25$
- etc.

9.2 Exercise Type 2: Computing MLE for Bigrams

Task: Calculate Maximum Likelihood Estimation for bigrams.

Corpus:

```
1. "I love machine learning"
2. "I love deep learning"
3. "Deep learning is great"
4. "Machine learning is fun"
```

Step-by-Step:

Step 1: Extract all bigrams

- From sentence 1: (I, love), (love, machine), (machine, learning)
- From sentence 2: (I, love), (love, deep), (deep, learning)
- From sentence 3: (Deep, learning), (learning, is), (is, great)
- From sentence 4: (Machine, learning), (learning, is), (is, fun)

Step 2: Count bigrams

- (I, love): 2
- (love, machine): 1
- (love, deep): 1
- (machine, learning): 1
- (deep, learning): 2
- (learning, is): 2
- (is, great): 1
- (is, fun): 1

Step 3: Count first words (for denominator)

- I: 2
- love: 2
- machine: 1
- deep: 2
- learning: 4
- is: 2

Step 4: Calculate probabilities

$$P(w_2|w_1) = \frac{C(w_1, w_2)}{C(w_1)}$$

Examples:

$$P(\text{love}|\text{I}) = \frac{C(\text{I, love})}{C(\text{I})} = \frac{2}{2} = 1.0$$
$$P(\text{machine}|\text{love}) = \frac{C(\text{love, machine})}{C(\text{love})} = \frac{1}{2} = 0.5$$
$$P(\text{deep}|\text{love}) = \frac{C(\text{love, deep})}{C(\text{love})} = \frac{1}{2} = 0.5$$
$$P(\text{is}|\text{learning}) = \frac{C(\text{learning, is})}{C(\text{learning})} = \frac{2}{4} = 0.5$$

9.3 Exercise Type 3: Computing MLE for Trigrams

Task: Calculate Maximum Likelihood Estimation for trigrams.

Corpus:

```
1. "I love machine learning"
2. "I love deep learning"
3. "Deep learning is great"
4. "Machine learning is fun"
```

Step-by-Step:

Step 1: Extract all trigrams

- From sentence 1: (I, love, machine), (love, machine, learning)
- From sentence 2: (I, love, deep), (love, deep, learning)
- From sentence 3: (Deep, learning, is), (learning, is, great)
- From sentence 4: (Machine, learning, is), (learning, is, fun)

Step 2: Count trigrams

- (I, love, machine): 1
- (I, love, deep): 1
- (love, machine, learning): 1
- (love, deep, learning): 1
- (deep, learning, is): 1
- (machine, learning, is): 1
- (learning, is, great): 1
- (learning, is, fun): 1

Step 3: Count bigram contexts (first two words)

- (I, love): 2
- (love, machine): 1
- (love, deep): 1
- (deep, learning): 1
- (machine, learning): 1
- (learning, is): 2

Step 4: Calculate probabilities

$$P(w_3|w_1, w_2) = \frac{C(w_1, w_2, w_3)}{C(w_1, w_2)}$$

Examples:

$$P(\text{machine}|\text{I, love}) = \frac{C(\text{I, love, machine})}{C(\text{I, love})} = \frac{1}{2} = 0.5$$
$$P(\text{deep}|\text{I, love}) = \frac{C(\text{I, love, deep})}{C(\text{I, love})} = \frac{1}{2} = 0.5$$
$$P(\text{learning}|\text{love, machine}) = \frac{C(\text{love, machine, learning})}{C(\text{love, machine})} = \frac{1}{1} = 1.0$$
$$P(\text{great}|\text{learning, is}) = \frac{C(\text{learning, is, great})}{C(\text{learning, is})} = \frac{1}{2} = 0.5$$
$$P(\text{fun}|\text{learning, is}) = \frac{C(\text{learning, is, fun})}{C(\text{learning, is})} = \frac{1}{2} = 0.5$$

9.4 Exercise Type 4: Computing Sentence Probability

Task: Given MLE probabilities, compute the probability of a sentence.

Given Bigram Probabilities (from previous example):

- $P(\text{love}|\text{I}) = 1.0$
- $P(\text{machine}|\text{love}) = 0.5$
- $P(\text{learning}|\text{machine}) = 1.0$ (assume)

Sentence: "I love machine learning"

Step-by-Step:

Step 1: Add sentence boundaries

```
<s> I love machine learning </s>
```

Step 2: Apply bigram formula

$$P(S) = P(\text{I}|\texttt{<s>}) \times P(\text{love}|\text{I}) \times P(\text{machine}|\text{love}) \times P(\text{learning}|\text{machine}) \times P(\texttt{</s>}|\text{learning})$$

Step 3: Plug in values (assuming some values for demonstration)

$$\begin{aligned} P(S) &= 0.5 \times 1.0 \times 0.5 \times 1.0 \times 0.25 \\ &= 0.0625 \end{aligned}$$

Important Notes:

- Always include `<s>` and `</s>` markers
- Multiply all conditional probabilities
- One zero makes entire probability zero

10 Word Meaning and Semantics

10.1 The Problem: N-grams Don't Understand Meaning

What's Missing from N-gram Models?

The Core Problem:
So far, we've treated words as just symbols:

- "cat" is just an index in a vocabulary: w_{42}
- "dog" is just another index: w_{17}
- We know $P(\text{dog}|\text{the})$ but we don't know that dogs and cats are related!

What's Wrong?

- N-grams can't tell that "car" and "automobile" mean the same thing
- They don't know "hospital" and "surgeon" are related
- Each word is completely independent
- We're missing the **meaning** of words!

Why Do We Care?
If our model understood meaning:

- Better predictions: "I drive my ___" could be "car" OR "automobile"
- Better generalization: Seeing "hospital" helps predict "surgeon"
- More human-like language understanding

10.2 Understanding Word Components

Three Key Concepts

1. Lemma = Dictionary Entry
The **lemma** is the base form of a word:

- "run", "running", "ran" → all have lemma "run"
- "mouse", "mice" → both have lemma "mouse"
- Think: "What would you look up in a dictionary?"

2. Sense = Actual Meaning
A **sense** is ONE specific meaning of a word.
Example: The word "bank" has multiple senses:

1. Financial institution: "I went to the bank to deposit money"
2. River edge: "We sat on the bank of the river"

Example: The word "mouse":

1. Animal: "The cat chased the mouse"
2. Computer device: "Click the mouse to open the file"

3. Polysemous = Multiple Meanings
A word is **polysemous** when it has multiple senses:

- "mouse" is polysemous (2 senses)
- "bank" is polysemous (2+ senses)
- "set" is VERY polysemous (100+ senses!)

Key Insight: One lemma can have many senses. When you see "mouse" in text, which sense is meant? That's called **word sense disambiguation**.

10.3 Synonyms: Words with Similar Meanings

Understanding Synonyms

Simple Definition: Synonyms are words that mean the same (or nearly the same) thing.
Examples:

- big / large → "a big house" = "a large house"
- couch / sofa → "sit on the couch" = "sit on the sofa"
- automobile / car → "I bought an automobile" = "I bought a car"
- filbert / hazelnut → same nut, different names

The Tricky Part: Perfect Synonyms Don't Really Exist!
Even "big" and "large" aren't always interchangeable:

- **Correct**: "my big sister" (older sister - natural!)
- **Wrong**: "my large sister" (sounds like she's physically large - unnatural!)

Why This Matters:

- Words carry subtle connotations and usage patterns
- Context determines which synonym is appropriate
- Some synonyms are formal (automobile) vs casual (car)

How Do We Measure Similarity?

The Question: Can we put a number on how similar two words are?
The Answer: Ask humans! That's what the **SimLex-999 dataset** does.
How It Works:

- Humans rate word pairs from 0 (totally different) to 10 (almost identical)
- 999 word pairs were rated
- Used to test if computers can match human judgments

Real Examples from the Dataset:

word1	word2	similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

What This Tells Us:

- 9.8: "vanish" and "disappear" are almost perfect synonyms
- 7.3: "behave" and "obey" are related but not identical
- 0.3: "hole" and "agreement" have basically nothing in common

Why This Dataset Is Useful:

- Benchmark to test word embedding models
- If a model's similarity scores match humans → model is good!
- Helps evaluate vector semantics (next topic)

10.4 Word Relatedness vs. Similarity

Two Different Concepts!

Similarity: Words that can substitute for each other
Examples:

- coffee / tea → both beverages, can substitute: "I'll have coffee" = "I'll have tea"
- dog / cat → both pets
- car / automobile → same thing

Relatedness: Words that appear together or are associated
Examples:

- coffee / cup → NOT similar! But strongly related (you drink coffee FROM a cup)
- doctor / hospital → NOT similar! But related (doctors work IN hospitals)
- knife / fork → NOT similar! But related (both are eating utensils)

Key Difference:

- Similar words can replace each other: "I need a {car/automobile}"
- Related words appear together: "Pour {coffee} into the {cup}"

Semantic Fields: Groups of Related Words

What is a Semantic Field?
A group of words that all relate to the same topic or domain. They tend to appear together in text!
Example 1 - Hospital Domain:
When you read a text about hospitals, you'll likely see:

- surgeon, nurse, doctor, patient
- scalpel, stethoscope, medicine
- operation, diagnosis, treatment

These words are **related** (they all belong to the medical field) but they're NOT **synonyms**!
Example 2 - Restaurant Domain:

- waiter, chef, customer
- menu, plate, fork, knife
- food, appetizer, dessert

Example 3 - House Domain:

- door, window, roof, floor
- kitchen, bedroom, bathroom
- family, furniture, bed

Why This Matters for Language Models:

- If you see "hospital" in text, "surgeon" becomes more likely to appear nearby
- This is called **co-occurrence**: related words appear together
- Modern models learn these relationships automatically from text!

10.5 Vector Semantics: The Solution!

From Words to Vectors

The Big Idea: Represent each word as a vector of numbers instead of just a symbol!
What We Had Before (N-grams):

- "cat" $= w_{42}$ (just an index)
- "dog" $= w_{17}$ (just another index)
- No relationship between them!

What We Have Now (Vectors):

- "cat" $= [0.2, -0.5, 0.8, 0.1, \ldots]$ (300 numbers)
- "dog" $= [0.3, -0.4, 0.7, 0.2, \ldots]$ (300 numbers)
- Vectors are close $\rightarrow$ words are similar!

The Magic: Similar words get similar vectors!

- "king" and "queen" $\rightarrow$ vectors are close
- "hospital" and "surgeon" $\rightarrow$ vectors are close
- "king" and "banana" $\rightarrow$ vectors are far apart

The Core Principle: "You Shall Know a Word by the Company It Keeps"

Famous Quote by Firth (1957): If two words appear in similar contexts, they probably have similar meanings!
Intuitive Example:
Sentence 1: "The **dog** is running in the park"
Sentence 2: "The **cat** is running in the park"
$\rightarrow$ "dog" and "cat" appear in similar contexts
$\rightarrow$ They probably have similar meanings!
$\rightarrow$ Give them similar vectors!
How It Works Automatically:

1. Read millions of sentences
2. For each word, look at what words appear nearby
3. Words with similar neighbors get similar vectors
4. Done! No human annotation needed!

Example - Learning "hospital" and "surgeon":
In text, you might see:

- "The surgeon works at the hospital"
- "She went to the hospital to see a surgeon"
- "Hospital staff including surgeons and nurses..."

$\rightarrow$ "hospital" and "surgeon" appear together often
$\rightarrow$ Give them related (but not identical) vectors

Visualizing Semantic Space

Imagine a 2D Map of Words:
In reality, we use 300+ dimensions, but imagine we squash it down to 2D:
Region 1 - Positive Sentiment:

- good, great, excellent, fantastic, wonderful, amazing, terrific, nice
- All cluster together in one area!

Region 2 - Negative Sentiment:

- bad, terrible, awful, horrible, worst, dislike
- All cluster together in a different area!

Region 3 - Function Words:

- to, by, that, the, a, is, are, with
- These cluster separately from content words!

What You Can Do:

- Measure distance: $\text{distance}(\text{good}, \text{great})$ is small
- Find similar words: What's near "king"? Answer: "queen", "monarch", "prince"
- Analogies: king - man + woman = queen (famous example!)

Modern Vector Models

Popular Approaches You Should Know:
1. Word2Vec (2013):

- Predict nearby words from center word (or vice versa)
- Very fast and efficient
- Still widely used!

2. GloVe (2014):

- Uses global co-occurrence statistics
- Captures both local and global context

3. Modern: BERT, GPT (2018+):

- Context-aware: "bank" has different vectors in "river bank" vs "money bank"
- Much more powerful but also more complex
- State-of-the-art for most tasks

Key Takeaway: Vector semantics solved the problem! Now words aren't just symbols - they carry meaning through their vector representations.

11 Quick Reference: Key Formulas

Essential Formulas Summary

N-gram Probabilities

Unigram:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i)$$

Bigram:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i|w_{i-1})$$

Trigram:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i|w_{i-2}, w_{i-1})$$

Maximum Likelihood Estimation

Bigram MLE:

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Trigram MLE:

$$P(w_i|w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

Smoothing

Add-1 (Laplace) Smoothing:

$$P_{\text{Add-1}}(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

Perplexity

Definition:

$$\text{PP}(W) = P(w_1, w_2, \ldots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\frac{1}{P(w_1, w_2, \ldots, w_N)}}$$

For Bigrams:

$$\text{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}}$$

Log Space:

$$\text{PP}(S) = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log P(w_i|w_1, \ldots, w_{i-1})\right)$$

Remember: Lower perplexity = Better model

Evaluation Metrics

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

12 Tips for Exam Success

Exam Strategy

For Unigram/Bigram/Trigram Counting:

1. Write down the entire vocabulary
2. Create a systematic counting table
3. Double-check your counts
4. Remember to include `<s>` and `</s>` if specified

For Probability Calculations:

1. Identify what type of model (unigram, bigram, trigram)
2. Write out the formula first
3. Substitute values carefully
4. Check if smoothing is required
5. Verify probabilities are between 0 and 1

For Sentence Probability:

1. Add sentence boundaries
2. Break into individual conditional probabilities
3. Look up or calculate each probability
4. Multiply them all together
5. Remember: one zero $\rightarrow$ entire probability is zero

For Perplexity:

1. Calculate sentence probability first
2. Count N (number of words, including special tokens)
3. Apply the perplexity formula
4. Lower value = better model

For Evaluation Metrics:

1. Draw the confusion matrix
2. Label TP, TN, FP, FN clearly
3. Write formulas before calculating
4. Show all steps

Common Pitfalls to Avoid:

- Forgetting to include special tokens `<s>` and `</s>`
- Confusing numerator and denominator in MLE
- Forgetting to add V to denominator in Add-1 smoothing
- Mixing up precision and recall definitions
- Calculating perplexity without taking the N-th root